

Epic / Code refactoring and architecture decisions review ▾

okay my tm8 sent me this as like a to do list or smthn i think most of it is already done but can you explain the things that arent done that need doing / if they actually do need doing and then explain some of the odd choices

features: change password remove message store - cpp - (empty)
reconfigure cpp to use stl and not just use qt frontend duplicate - ? -
resolve - whats backend/frontend/ stubs int here for forwarding - change

Show more

Searched memory >

Good, I've got a thorough picture of the codebase. Let me now break down each item clearly.

The To-Do List, Item by Item

✓ Change Password — Done

Fully implemented end to end. Backend has `POST /api/auth/change-password` in `auth.py` with rate-limiting, constant-time password verification, a new-password must-differ check, and session rotation (new JWT + CSRF token issued on success). The Qt client has `onChangePasswordClicked()` in `MainWindow.cpp`, a proper dialog, a 12-char minimum check, and the correct ordering logic (server change first, then `rekeyIdentity` on the daemon so they can never desync in the bad direction). Nothing to do here.

✓ Remove MessageStore (C++ empty stubs) — Done, and the framing is wrong

The note says "remove MessageStore — empty." It's actually not empty at all — it's a fully implemented, well-documented class using `std::unordered_map`, `std::unordered_set`, `std::find_if`, `std::any_of`, `std::copy_if`, `std::count_if`, and a custom case-insensitive `std::sort`. There's a `// TODO: Add toJson() / fromJson()` comment in `Message.cpp` but that's a stub comment on an intentionally deferred

helper, not a missing feature. **Don't touch MessageStore — it's a showcase piece for Burkley/Kashif's C++ criteria.**

✅ Reconfigure C++ to use STL not just Qt — Already done

`MessageStore` already uses pure STL containers internally with Qt only at the signal boundary (`emit messageAdded()`), which is exactly the right design. The comment in the header makes this explicit. No action needed.

⚠️ Frontend Duplicate — Partially an issue

There are actually two separate frontends:

- `frontend/` — the more complete version (has real X3DH send/receive, WebSocket, TOFU pinning, revoke, delete, forward, download)
- `backend/frontend/` — an older/simpler version served by FastAPI, with `sendMessage()` and `fetchInbox()` as TODOs

The "duplicate" note is probably referring to this. The backend-served frontend has `// TODO: implement once crypto.js encryption is complete` stubs. This isn't something you need to fully resolve before submission — the working frontend is the one in `frontend/`. You should just be able to explain the situation: the backend-served frontend is a secondary thin client that shares the same API but has incomplete crypto integration.

⚠️ Fix TCP/Unix Socket Mismatch — Low priority, defensible as-is

The note correctly identifies this: the Python daemon's `README.md` and its own `crypto-daemon/README.md` claim it uses a Unix domain socket (or named pipe on Windows), but `transport.py` actually uses a TCP loopback socket on `127.0.0.1:47291`, and so does `CryptoDaemonClient.cpp`. So the mismatch is between the docs and reality — the code itself is actually internally consistent (both sides use TCP).

The suggestion to switch to `QLocalSocket` is the "correct" fix in principle (Unix sockets are slightly faster and more appropriate for IPC), but in practice the TCP loopback approach is functionally identical in security terms since it never leaves the loopback interface, and it works cross-platform without any platform-specific code. The comment in `transport.py` even explains this reasoning explicitly.

What to say if asked: "We chose TCP loopback over a Unix socket because it's fully cross-platform — the same transport code runs on Windows and Linux without conditional compilation. The socket is bound to `127.0.0.1` and never reachable from the network, so security properties are equivalent." Update the README to match reality if you haven't already — that's the only real action item here.

The "Odd Choices" to Defend

`shared_ptr` everywhere instead of `unique_ptr`

Looking at the code, `unique_ptr` is actually used in places (e.g. `m_socket` in `CryptoDaemonClient` is `std::unique_ptr<QTcpSocket>`). The `shared_ptr` usage is probably in `Client` holding the daemon. The honest answer for interview is:

"We used `shared_ptr` in cases where ownership semantics weren't fully clear at design time, specifically where the daemon client needed to be passed into multiple objects. `unique_ptr` is preferable where single ownership is clear and we use it there — for example the socket in `CryptoDaemonClient`. The spec says 'use `unique_ptr` or `shared_ptr` where appropriate', not 'always prefer unique.' If pressed, we'd acknowledge that some of those could be refactored to `unique_ptr` with clear ownership transfer, but `shared_ptr` is safe and correct."

OPK key IDs based on Unix timestamp, not globally unique

The code in `Client.cpp` is explicit about this: SPK key ID = `currentSecsSinceEpoch()`, OPK key IDs = `spkKeyId * 100 + i`. The comment even says the write-off assumption: "assume no use will create two accounts within one second."

The honest defence: "OPK key IDs only need to be unique per-user on the server — there's no global uniqueness requirement in the X3DH spec. The timestamp-based scheme guarantees uniqueness as long as a user doesn't upload two batches within the same second, which is guaranteed by the registration flow (one batch on register, one on login). We acknowledged this as a deliberate simplification."

Private key in `.env` — "what if the server is compromised?"

The answer your teammate wrote is the right one: an attacker who gets the server's private key can write arbitrary digests to the Sepolia contract going forward, but all *existing* records are immutable on-chain and can still be verified against their original

timestamps. The blockchain component's integrity guarantee is retrospective — past records can always be independently verified. The correct defence: "Compromise of the server wallet key is a known limitation we've documented. It's equivalent to compromising a Certificate Authority's signing key in a PKI system — it affects future trust, not historical records. Mitigation would be HSM storage or a multi-sig contract, which is out of scope for this project."

Summary of Actual Remaining Actions

Item	Status	Action
Change password	Done	Nothing
MessageStore stubs	Done (misleading note)	Nothing — keep it
STL vs Qt	Done	Nothing
Frontend duplicate	Partially	Be able to explain the two-frontend situation
TCP/Unix socket mismatch	Mismatch is docs vs code, not C++ vs Python	Update README to say TCP loopback, not Unix socket
<code>shared_ptr</code> everywhere	Defensible	Prep the explanation above
OPK timestamp IDs	Defensible write-off	Prep the explanation above
Private key in <code>.env</code>	Documented limitation	Prep the explanation above

The only concrete action before submission is fixing the README/docs to accurately describe the TCP transport so the docs match the code.

excerpt_from_previous_claude_message.txt
4 lines

TXT

is this E2E secure? it sure doesnt sound like it... could an attacker take advantage of this

Searched project for “OPK one-time prekey key_id unique constraint database” >

Good — now I have the full picture. Let me give you a precise answer.

Short answer: no, this does not break E2E security. The key_id collision risk is real but the consequences are limited and don't affect confidentiality. Here's the precise breakdown:

What key_id actually does

The key_id is a label, not a cryptographic value. It's used for one thing: when Alice sends Bob a message, she tells Bob *which* OPK she used (by key_id) so Bob can look up the corresponding private key and complete the X3DH. The actual security comes from the Diffie-Hellman computation using the public key bytes themselves — not the ID.

What a collision actually causes

If two batches produce the same key_id (same Unix second, same slot index), the DB has no unique constraint on `(user_id, key_id)` for OPKs — it would just insert a second row with the same key_id. When Alice requests Bob's keybundle, the server picks the first unused OPK with `WHERE used = False LIMIT 1`. It selects by DB row order, not key_id, so a duplicate key_id doesn't cause the wrong OPK to be served.

The actual failure mode is: if Bob receives a message referencing key_id `x`, and he has two OPKs with that ID in his on-disk store, his daemon looks up the OPK by its **public key bytes** (see `session_store.py` — the filename is derived from the OPK's public key b64, not its key_id). So the lookup is actually keyed on the public key, not the key_id. A key_id collision is just a metadata oddity.

Worst realistic outcome of a collision: confusion or a failed X3DH on that one message — not decryption of a different session, not key exposure.

Is E2E confidentiality affected?

No. Even in the collision case:

- The OPK private key never leaves the daemon
- The shared secret SK is derived from DH operations on the actual key bytes

- A wrong `key_id` lookup → failed `x3dh_receive` → error, not silent decryption with a wrong key
- No attacker can exploit this without already having access to Bob's device

The honest interview framing

What you said in my previous message was actually slightly wrong on one point — I said "one batch on register, one on login" guarantees no same-second uploads, but actually the startup flow uploads a *new* batch on every login (unless `freshRegistration` is true). If a user logs in and out rapidly you could theoretically get same-second uploads. Still harmless for the reasons above, but the framing should be:

"`key_id` is a lookup hint, not a cryptographic input. A collision would at worst cause a failed session establishment on a single message. Confidentiality relies on the DH computation over the actual key bytes, which are unique by construction — libsodium's X25519 key generation uses a CSPRNG. We acknowledged the ID scheme as a simplification and documented it."

That's accurate and defensible with Eoin.

Okay, now I want to make those fixes that we loisted above , smthn to do with a read me and deleting a frontend stub thing

Searched memory >

Good, I have everything I need. Two files to fix:

1. `crypto-daemon/README.md` — claims Unix domain socket / named pipe, actually TCP loopback
2. `crypto-daemon/main.py` docstring — same false claim at the top
3. `backend/frontend/app.js` — the stub `sendMessage` and `fetchInbox` TODOs (the incomplete second frontend)

Let me read the exact current content of each before editing.

Ran 3 commands >

The project files in the knowledge base are PDFs only — the actual source files are on the VM/repo, not mounted here. So I can't edit them directly. I'll produce the exact diffs

for you to apply. Let me draft the precise changes.

Fix 1: `crypto-daemon/README.md`

Replace the intro paragraph and the Running section. The lies are in lines 3, 9-12, and the "Default address" block.

Find this (current incorrect text):

A local Python process that handles every cryptographic operation for the SecureMsg client – identity keys, X3DH key agreement, the Double Ratchet, and at-rest key/session encryption. The C++ Qt UI never touches a private key; it talks to this daemon over a Unix domain socket (or named pipe on Windows) using line-delimited JSON.

Replace with:

A local Python process that handles every cryptographic operation for the SecureMsg client – identity keys, X3DH key agreement, the Double Ratchet, and at-rest key/session encryption. The C++ Qt UI never touches a private key; it talks to this daemon over a TCP loopback socket (127.0.0.1:47291) using newline-delimited JSON.

Find this (Running section):

```
## Running
```

```
pip install -r requirements.txt
```

```
python main.py          # bind to default address
```

```
python main.py --address /tmp/securemsg-crypto.sock # POSIX
```

```
python main.py --address \\.\pipe\securemsg-crypto # Windows
```

Default address:

- POSIX: ``/tmp/securemsg-crypto.sock` (AF_UNIX stream, mode 0600)`
- Windows: ``\\.\pipe\securemsg-crypto` (named pipe via `multiprocessing.connection`)`

Replace with:


```
## Running
```

```
pip install -r requirements.txt
```

```
python main.py          # bind to default 127.0.0.1:47291
```

```
python main.py --address 127.0.0.1:47292 # second instance (two-client testing)
```

Default address: `127.0.0.1:47291` – TCP loopback, never reachable from the network.

The C++ client connects via `QTcpSocket`; pass `-DDAEMON\_PORT=47292` at CMake configure time to bind a second client instance to a second daemon.

Find this (Wire protocol section):

One JSON object per message. On POSIX each request is a single line ending in `\n`; on Windows each request is one length-prefixed `send\_bytes` frame (stdlib `multiprocessing.connection` handles framing).

Replace with:

One JSON object per message. Each request is a single line ending in `\n`, on both POSIX and Windows. The TCP framing is identical on all platforms.

Fix 2: `crypto-daemon/main.py` **docstring**

Find this (top of file docstring):

Accepts connections on a Unix domain socket (POSIX) or named pipe (Windows), reads framed JSON requests, dispatches them through `handlers.handle()`, and writes the JSON response back on the same connection. One connection may issue many sequential requests.

Replace with:

Accepts connections on a TCP loopback socket (127.0.0.1:47291 by default), reads newline-framed JSON requests, dispatches them through `handlers.handle()`, and writes the JSON response back on the same connection. One connection may issue many sequential requests.

Also fix the `--address` help string in the same file:

Find:

python

```
help="socket path (POSIX) or pipe name (Windows); default is platform st
```

Replace:

python

```
help="TCP address as host:port (default: 127.0.0.1:47291)",
```

Fix 3: `backend/frontend/app.js` stubs

This is the incomplete second frontend served by FastAPI. The two stub functions should either be removed or clearly marked. Cleanest option — replace the fake stubs with honest placeholders that won't confuse anyone reading the code:

Find:

javascript

```
async function sendMessage() {  
  // TODO: implement once crypto.js encryption is complete  
  console.warn("sendMessage: crypto.js not yet implemented");  
}  
  
async function fetchInbox() {  
  // TODO: implement once crypto.js decryption is complete  
}
```

Replace with:

javascript

```
async function sendMessage() {  
  // Not implemented in this client – use the standalone frontend/ for full E2EE  
  console.warn("sendMessage: not implemented in the backend-served client");  
}  
  
async function fetchInbox() {
```

```
// Not implemented in this client – use the standalone frontend/ for full E2EE  
}
```

That's all three changes. They're all small targeted edits — grab them one file at a time and apply in your editor or with `sed`. Want me to write a PowerShell script that applies all three via the SSH connection to the VM?

im ju7st going to get claude code to do it - give me the prompts i shud send it

Three separate prompts, one per file:

Prompt 1 — `crypto-daemon/main.py` docstring

In `crypto-daemon/main.py`, fix the module docstring at the top of the file. It currently claims the daemon uses "a Unix domain socket (POSIX) or named pipe (Windows)". Replace that with "a TCP loopback socket (127.0.0.1:47291 by default)". Also fix the `--address` argparse help string from "socket path (POSIX) or pipe name (Windows); default is platform standard" to "TCP address as host:port (default: 127.0.0.1:47291)". No other changes.

Prompt 2 — `crypto-daemon/README.md`

In `crypto-daemon/README.md`, fix three places where the docs incorrectly claim the daemon uses a Unix domain socket or named pipe. The actual transport is TCP loopback on 127.0.0.1:47291 on all platforms.

1. Opening paragraph: change "it talks to this daemon over a Unix domain socket (or named pipe on Windows) using line-delimited JSON" to "it talks to this daemon over a TCP loopback socket (127.0.0.1:47291) using newline-delimited JSON"

2. Running section: replace the two example commands that show a Unix socket path and Windows pipe path with "python main.py --address 127.0.0.1:47292" as the custom-address example. Replace the "Default address" bullet list

(POSIX/Windows) with a single line: "Default address: 127.0.0.1:47291 – TCP loopback, never reachable from the network."

3. Wire protocol section: replace "On POSIX each request is a single line ending in \n; on Windows each request is one length-prefixed send_bytes frame (stdlib multiprocessing.connection handles framing)" with "Each request is a single line ending in \n on both POSIX and Windows. The TCP framing is identical on all platforms."

No other changes.

Prompt 3 — `backend/frontend/app.js` stubs

In `backend/frontend/app.js`, find the two stub functions `sendMessage` and `fetchInbox` that have `TODO` comments saying "implement once crypto.js encryption/decryption is complete". Replace their bodies with a comment saying "Not implemented in this client – use the standalone frontend/ directory for full E2EE messaging." Remove the `console.warn` from `sendMessage`. No other changes.

shud i just delete the folder? frontend/

No — that's the **good** frontend, the one with real X3DH, WebSocket, TOFU pinning, revoke, delete, forward, download. Don't touch it.

The incomplete one is `backend/frontend/` — that's the one served directly by FastAPI. That's what you'd consider deleting.

But I'd leave it too, honestly. It's harmless, and deleting it might break a static file route in the FastAPI app that the examiners hit when they browse to the backend URL. The stub comment fix is cleaner — makes it obvious the real client is `frontend/`.

i would rather just get rid of it if its not used anywhere..

Searched project for “backend frontend static files mount serve fastapi” >

Don't delete it. Found the smoking gun — in `backend/Dockerfile`:

```
dockerfile
```

```
COPY frontend/ /app/frontend/
```

And in `backend/main.py`:

```
python
```

```
app.mount("/static", StaticFiles(directory="/app/frontend"), name="static")
```

The `frontend/` folder (the good one) gets copied into the Docker image and served at `/static`. So `backend/frontend/` is separate and genuinely unused — but if you delete `backend/frontend/` you're fine. If you accidentally deleted `frontend/` at the repo root, the Docker build would break.

Claude Code prompt:

```
Delete the directory backend/frontend/ and all its contents. It is an unused duplicate of the frontend/ directory. The actual frontend is served from the repo-root frontend/ folder which gets copied into the Docker image. Do not touch frontend/ at the repo root. After deleting, confirm no imports or references to backend/frontend/ exist anywhere in the codebase.
```

i might just get claude to read that folder and the actual frontend folder?
and make sure we arent deleting anything that should be in the final code

Good call. Claude Code prompt:

Compare the contents of backend/frontend/ and frontend/ (repo root). For every file in backend/frontend/, check whether it contains anything – any function, logic, or feature – that is not present in frontend/. List what is unique to backend/frontend/ if anything. Do not delete or modify anything yet, just report.

I've completed the comparison. Here's the report.

Comparison: backend/frontend/ vs frontend/

Both directories contain the same six files. Four are byte-for-byte identical:

- [index.html](vscode-webview://1t5i1480760aconv518l0ab9gmj9psjatpvkqbqse38u1audf6de

Show more

Yes, safe to delete. Give Claude Code:

Delete the directory backend/frontend/ and all its contents.

That's it. Nothing in there that isn't better in frontend/.

can i just do it myself

Yeah, just run this in PowerShell from the repo root:

```
powershell  
  
Remove-Item -Recurse -Force backend\frontend
```

Then commit it.

Deleted [backend/frontend/](vscode-webview://1t5i1480760aconv518l0ab9gmj9psjatpvkbbqse38u1audf6de8/backend/frontend/) and all six files. Since this is a git repo, I used `git rm -r`, which removes them from both disk and the index — the deletions are now staged. They'll be finalized on your next commit (let me know if you'd like me to commit). Confirmed earlier that nothing unique was lost.

the first 2 prompts ran safely too - i think we are all good

Perfect. So the three fixes are done:

1. `crypto-daemon/main.py` — docstring and argparse help corrected to TCP loopback
2. `crypto-daemon/README.md` — Unix socket / named pipe references replaced with accurate TCP docs
3. `backend/frontend/` — deleted, nothing lost

Commit those and you're clean. Good timing too — less than 24 hours to submission.

give me cmds to commit and push onto a new branch

powershell

```
git checkout -b fix/docs-and-cleanup
git add -A
git commit -m "fix: correct daemon transport docs and remove stale backend/frontend"
git push -u origin fix/docs-and-cleanup
```

Then open a PR into main on GitHub when you're ready.

okay all done - now, just some things I want to do now are get claude code to run through each part of the rubric and see how the codebase is - i also need to try deploy the web page / blockchain page i think? or i atleast want to test them locally - my tm8 said theyve been pushed - but yh

Two separate things — let me give you the Claude Code prompt for the rubric check first, then we'll deal with the frontend/blockchain testing.

Claude Code rubric audit prompt:

Read the following files to understand the assessment rubric, then audit the codebase against each criterion and give me a honest gap analysis with a score estimate for each section:

Rubric files to read first:

- CS4455Epic2026_1.pdf (C++ / Networks rubric)
- CS4455_Cryptography_EPIC_Guide.pdf (Crypto rubric)
- ComputerSystemsandCyberSecurityEPIC.pdf (Cyber Security rubric)
- BlockchainEPICGuide.pdf (Blockchain rubric)

Then audit the actual code. For each rubric criterion, state:

1. What the rubric requires
2. What the codebase does
3. Any gaps or weaknesses
4. Honest score estimate

Be critical — do not be generous. Flag anything an examiner could penalise.

For the frontend/blockchain testing — what exactly did your teammate push? Is it:

- The `frontend/` folder (the web client served statically)
- A blockchain verification page (`verify.html` / `verify.js`)
- The deployed smart contract on Sepolia

Do you know if the contract address is already set in the `.env` on the VM? That's the key thing — if `CONTRACT_ADDRESS` isn't set the blockchain integration silently does nothing.



anytyhing done by juliahooper

Julia's commits today:

- **feat/secure-web-client** (merged PR #32) — "add Load older messages button and update styles", "implement key wrapping and unwrapping for enhanced security", and notably "**Remove frontend files for SecureMsg application**" (f4736d1) which sounds alarming but was probably clearing out old files before adding the new ones in the same branch
- **post-pentest-1-edits** (merged PR #34 via Holly's branch) — security fixes from the automated pentest

The one I'd check is that "**Remove frontend files**" commit (f4736d1) — given you just deleted `backend/frontend/` today and Julia also removed frontend files 3 hours ago, worth making sure there's no conflict or double-deletion weirdness on main.

Quick thing to run:

```
powershell  
git log --oneline frontend/ -5
```

and

```
powershell  
git show f4736d1 --stat
```

That'll tell you exactly which files Julia removed. If `frontend/index.html`, `frontend/app.js` etc are still present on main you're fine.

```
PS C:\Users\m4tth\EPIC> git log --oneline frontend/ -5
fatal: option '-5' must come before non-option arguments
PS C:\Users\m4tth\EPIC>
```

powershell

```
git log -5 --oneline frontend/
```

```
PS C:\Users\m4tth\EPIC> git log -5 --oneline frontend/
ab78a44 (origin/fix/dos-skipped-keys, fix/dos-skipped-keys) fix: add
MAX_SKIPPED pre-checks before skip loops in DR and frontend
2d1f6f4 feat: implement X3DH + Double Ratchet crypto layer for web
client
7a75a90 feat: implement Content Security Policy for static files and add
verification logic to new JavaScript file
99240b5 feat: add frontend static files and update API_BASE to use
dynamic origin
db26d2a fix: update API_BASE to point to the correct server address
PS C:\Users\m4tth\EPIC>
```

That's fine — `frontend/` is intact and has real commits including the X3DH/Double Ratchet work. Julia's "remove frontend files" commit was almost certainly removing the `backend/frontend/` copies, which lines up with what you just did too.

Now run this to confirm the key files are actually there:

powershell

```
git show HEAD:frontend/app.js | Select-String "x3dh\|ratchet\|sendMessage" | Sel
```

If that returns matches you know the good frontend is on main and untouched.

no output

That means either the regex didn't match or the file isn't on HEAD yet. Try:

```
powershell
```

```
git show HEAD:frontend/app.js | Select-String -Pattern "sendMessage"
```

and

```
powershell
```

```
ls frontend/
```

```
PS C:\Users\m4tth\EPIC> git show HEAD:frontend/app.js | Select-String -
Pattern "sendMessage"
async function sendMessage() {
  console.error("sendMessage failed:", err);
  document.getElementById("btn-send").addEventListener("click",
sendMessage);
  if (e.key === "Enter" && !e.shiftKey) { e.preventDefault();
```

Show more

All good — `frontend/` is intact with all 6 files and `sendMessage` is properly implemented (not a stub). You're fine there.

Now the deployment question — the frontend is actually already being served by your backend. The Dockerfile copies `frontend/` into the image and FastAPI mounts it at `/static`. So if the VM is running, it's already deployed at:

```
https://alpha-and-the-cryptmunks.theburkenator.com/static/index.html
```

Try that in your browser now. If it loads you're good.

The blockchain/verify page would be at:

```
https://alpha-and-the-cryptmunks.theburkenator.com/static/verify.html
```

But for those to work the VM needs to be running the latest image. When did you last rebuild on the VM? If Julia's secure-web-client changes haven't been deployed yet you'll need to SSH in and do:

```
bash
```

```
git pull && docker compose up --build -d api
```

those urls gave me 502 errors will i run this?:
git pull && docker compose up --build -d api

502 means the container is down or crashed, not a routing issue. Yes, but you're on Windows so:

```
powershell
```

```
ssh student@200.69.13.70 -p 2202
```

Then once you're in:

```
bash
```

```
cd EPIC && git pull && docker compose up --build -d api
```

```
student@alpha-and-the-c  
ryptmunks:~$ cd EPIC &&  
git pull && docker compos  
e up --build -d api remote:  
Enumerating objects: 68,  
done. remote: Counting ...
```

PASTED

All good — container started cleanly. Now check the frontend loads:

```
https://alpha-and-the-cryptmunks.theburkenator.com/static/index.html
```

And verify page:

```
https://alpha-and-the-cryptmunks.theburkenator.com/static/verify.html
```

Try both in your browser now.

yeah im still getting a 502

The container started but Caddy (the reverse proxy) might not be routing to it yet, or the app crashed on startup. Check the logs:

```
bash
docker compose logs --tail=50 api
```

```
student@alpha-and-the-c
rypmunks:~/EPIC$ docke
r compose logs --tail=50 a
pi83e98253349ac74c7774
a97d0caa5e61f1e 0.1s WA
RN[0000] /home/stude...
```

PASTED

Missing dependency — `cryptography` isn't in `requirements.txt`. Quick fix:

```
bash
grep cryptography backend/requirements.txt
```

If it's not there, add it:

```
bash
echo "cryptography" >> backend/requirements.txt
git add backend/requirements.txt
git commit -m "fix: add missing cryptography dependency to requirements.txt"
git push
```

Then back on the VM:

```
bash
git pull && docker compose up --build -d api
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ grep cryptography
backend/requirements.txt      PyJWT[cryptography]>=2.8.0
# JWT encode/decode          student@alpha-and-
the-cryptmunks:~/EPIC$
```

It's there but as an extras dependency of PyJWT, not a standalone install — so `from cryptography.exceptions import InvalidSignature` can't find it. Fix it by adding it explicitly:

```
bash
echo "cryptography>=42.0.0" >> backend/requirements.txt && git add backend/requi
```

wait i just ran that on the VM

That's fine actually — the commit will be on main from the VM which is a bit messy but it works. Run:

```
bash
docker compose logs --tail=20 api
```

to see if it starts cleanly this time.

```
these could be old i went and got lunch.:
student@alpha-and-the-cryptmunks:~/EPIC$ docker compose logs --
tail=20 api                                WARN[0000]
/home/student/EPIC/docker-compose.yml: version is obsolete
api-1 | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
api-1 | File "/usr/local/lib/python3.12/site-packages/uvicorn/importer.py",
line 22 in import_from_string          api-1 | raise exc from None
```

[Show more](#)

Same problem, different missing package. Run:

```
bash
```

```
echo "python-jose[cryptography]>=3.3.0" >> backend/requirements.txt && docker co
```

No need to commit yet — let's just get it running first, then we'll do one clean commit with all missing deps once it boots.

theres prolly a bunch of stuff no..?

Yeah likely. Rather than whack-a-mole one by one, do this:

```
bash
```

```
docker compose exec api pip freeze > /tmp/installed.txt && docker compose exec a  
with open('main.py') as f: src = f.read()  
"
```

Actually simpler — just run this to check all imports at once:

```
bash
```

```
docker compose exec api python -m py_compile main.py && echo "OK"
```

If that fails, then just let pip tell you everything that's actually missing in one shot:

```
bash
```

```
docker compose exec api pip check
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ docker compose exec api  
python -m py_compile main.py && echo "OK"  
WARN[0000] /home/student/EPIC/docker-compose.yml: version is
```

```
obsolete [Errno 13] Permission denied:
'__pycache__/_main.cpython-312.pyc.139227183875536'student@alpha-
and-the-cryptmunks:~/EPIC$
```

Permission issue running as non-root. Try:

```
bash
docker compose exec api pip check
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ docker compose exec api
pip check
WARN[0000] /home/student/EPIC/docker-compose.yml: version is
obsolete No broken requirements found.
student@alpha-and-the-cryptmunks:~/EPIC$
```

That means the packages that are installed are fine, but `jose` isn't installed at all so pip doesn't even know it's missing. The issue is Julia's new code added imports that aren't in `requirements.txt`.

Quickest fix — just try to start and catch all the missing modules in one build:

```
bash
docker compose exec api python -c "from app.routers import auth, messages, block"
```

That'll either work or dump all the missing imports at once.


```
student@alpha-and-the-cryptmunks:~/EPIC$ docker compose exec api
python -c "from app.routers import auth, messages, blockchain, ws" 2>&1
WARN[0000] /home/student/EPIC/docker-compose.yml: version is
obsolete                                     Traceback (most recent call last):
File "<string>", line 1, in <module>
File "/app/app/routers/ws.py", line 19, in <module>
from jose import JWTError, jwt
ModuleNotFoundError: No module named 'jose'
student@alpha-and-the-cryptmunks:~/EPIC$
```

Only one missing so far. Add it and rebuild:

```
bash
```

```
echo "python-jose[cryptography]>=3.3.0" >> backend/requirements.txt && docker co
```

That adds it, rebuilds, waits 15 seconds, then immediately tests all imports so we find the next missing one if there is one.

wait now what sorry it just rebuilt

Run the import test:

```
bash
```

```
docker compose exec api python -c "from app.routers import auth, messages, block
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ docker compose exec api
python -c "from app.routers import auth, messages, blockchain, ws" 2>&1
WARN[0000] /home/student/EPIC/docker-compose.yml: version is
obsolete                                     student@alpha-and-the-
cryptmunks:~/EPIC$
```

Clean — no errors. Check if the API is actually up now:

```
bash
```

```
curl http://localhost:8000/health
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ curl  
http://localhost:8000/health  
{\"status\":\"ok\"}student@alpha-and-the-cryptmunks:~/EPIC$
```

API is up. Try the frontend in your browser now:

```
https://alpha-and-the-cryptmunks.theburkenator.com/static/index.html
```

okay, yk what i might just leave this for my tm8 its meant to be her job

Yeah fair enough — it's up and healthy, that's the important bit. Leave the frontend/blockchain presentation layer to Julia.

What do you want to focus on now — the rubric audit, or something else before the 5pm deadline tomorrow?

